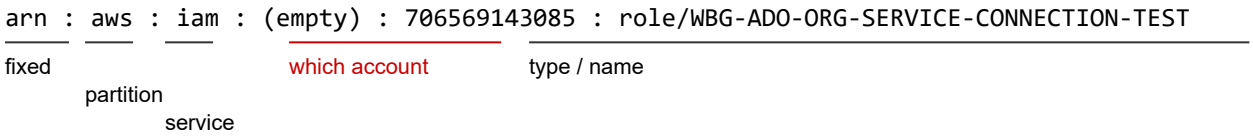
AWS identity and credentials: who can talk to AWS, and with what proof

Domain 1 of the ACN-2026-31070 case. Sections 1 to 5 are the five parts, in build order. Sections 6 to 10 compare, widen, and give you a request list. Every part has the same shape: what it is, a real example, how you check it as an auditor, and the two mistakes people make. Controls S1, S2, A3, A5 and D1 rest on this page.

1. AWS account

An **account** is the box that holds your AWS resources and the identities that can use them. It has a **12 digit account ID**, such as `706569143085` in this case. [3]

Every thing in AWS has an **ARN** (Amazon Resource Name), its full address. The pattern is `arn:partition:service:region:account-id:resource`. IAM objects have no region, so that slot is empty. ARNs are not secret. [3]



Does not apply when: the partition is not `aws`. China and GovCloud use `aws-cn` and `aws-us-gov`. [3]

How you check it: read the ARN left to right. The account ID tells you who owns the role. The evidence in this case shows `assumed-role/WBG-ADO-ORG-SERVICE-CONNECTION-TEST/aws-vsts-tools` in account `706569143085`. If that ID is not the account the team told you about, you stop and ask.

Two mistakes: treating an ARN as a secret and refusing to share it; and reading only the role name and missing that the account ID is different.

2. IAM user and access keys

An **IAM user** is an identity with long-term credentials. The usual pair is an **access key ID** and a **secret access key**. They work until someone deletes or deactivates them, from anywhere. Whoever holds them *is* that user. [2]

AWS says to use users only where federation cannot do the job: a workload that cannot use a role, a third-party client, or CodeCommit. [1] If you must keep a user, it should have MFA, and keys should be updated and removed using the access key last used information. [2]

Does not apply when: the caller is a role. Roles have no password and no keys. [1]

How you check it: ask for the **credential report**. It is a CSV listing every IAM user and the status of passwords, access keys and MFA. IAM builds a new one at most every four hours. The permissions are `iam:GenerateCredentialReport` and `iam:GetCredentialReport`. [4]

```
aws iam generate-credential-report
aws iam get-credential-report
```

Read the columns for the user the pipeline used to have. A **passing** result for control S1 is the key marked inactive or the user gone. A **failing** one is an active key with a recent last-used date. The report covers only

the first two keys per user, so also run `ListAccessKeys` for the full list. [4]

Two mistakes: believing a key is safe because "nobody uses it" (it is valid until deleted); and checking IAM but not the places a key can hide, such as variable groups, YAML, scripts, repos and agents, which is what control S1 lists.

3. IAM role

A **role** is an identity with permissions but no standard password or keys. Anyone allowed to **assume** it gets temporary credentials for a **session**. [1] A new account has no roles; services may add **service-linked roles** later. [1]

Two policies sit on a role. The **trust policy** says who may assume it. The **permission policy** says what it can do once assumed. Control A3 audits the first; control A5 audits the second.

Example. The role `WBG-ADO-ORG-SERVICE-CONNECTION-TEST` is assumed by the Azure DevOps service connection. In the log it appears as `arn:aws:sts::706569143085:assumed-role/WBG-ADO-ORG-SERVICE-CONNECTION-TEST/aws-vsts-tools`. The word `assumed-role` proves a role was used, not a user. The last part, `aws-vsts-tools`, is the **session name**, a label for that borrowing. [5]

Does not apply when: you need a credential that never expires. A role cannot give one.

How you check it: in the console, open IAM, then Roles, then the role. Read the **Trust relationships** tab and the **Permissions** tab. Passing for A3: only the Azure DevOps provider may assume it. Failing: a wildcard principal, or any account that should not be there. Writing the finding: "Role X trusts principal Y, which allows Z to assume it; restrict the trust policy to the Azure DevOps provider."

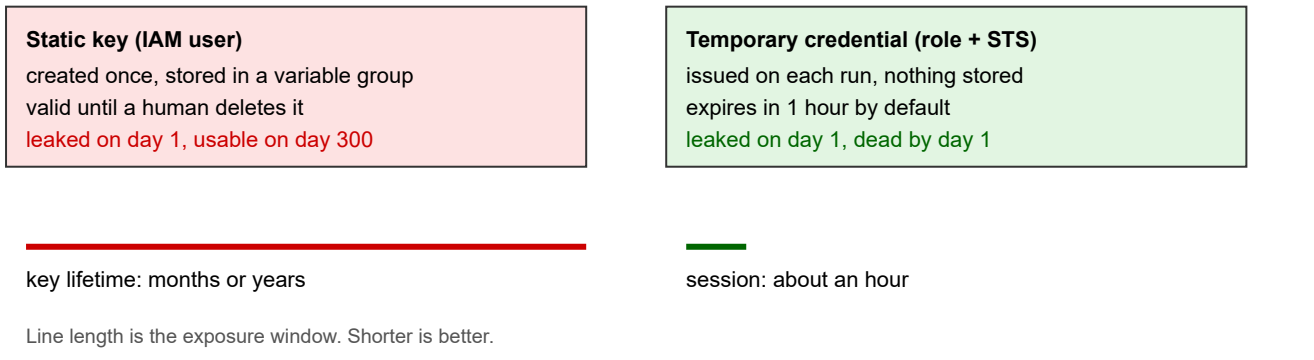
Two mistakes: auditing only permissions and not the trust policy; and thinking the role's *name* proves who can use it. Only the trust policy does.

4. STS and temporary credentials

STS (Security Token Service) is the AWS service that hands out temporary credentials. [5] They are three values: an access key ID, a secret key and a **session token**. They expire. For `AssumeRoleWithWebIdentity` the default is one hour. The session can be set from 15 minutes up to the role's maximum, which is 1 to 12 hours. [5]

This is the call this case uses. It takes the token from the pipeline and the role ARN, and **needs no AWS credentials to make**. That is why no key has to be stored. [5]

```
Pipeline --(OIDC token + role ARN)--> STS
STS --(access key ID, secret key, session token, expiry)--> Pipeline
```



Does not apply when: a long-term key is used. A plain access key has no session token and no expiry.

How you check it: look at the role's maximum session duration in the role summary, and the log lines showing "Assuming role via OIDC Token". Control S2 asks that none of these values appear in logs, outputs or artifacts. Search the logs for the session token. Control D1 asks that changes to the duration be reviewed.

Two mistakes: assuming "temporary" means "safe" (a leaked token works until it expires, so shorter is better); and forgetting the role's maximum can cap the duration you request.

5. Static versus temporary: the why

A leaked static key stays valid until a human finds and kills it. A leaked temporary credential dies on its own. That is the whole argument. AWS recommends temporary credentials with roles for workloads, and long-term keys only where nothing else works. [2]

For a pipeline running outside AWS, AWS lists these ways to get temporary credentials: IAM Roles Anywhere with an X.509 certificate, `AssumeRoleWithSAML`, and `AssumeRoleWithWebIdentity` with a JWT from an identity provider. [2] This case uses the last one, with Azure DevOps as the provider. [6] [7]

Does not apply when: the workload truly cannot use any of these. Then the key is an exception and must be reviewed, rotated and justified. [2]

How you use it: in a finding, quote the AWS recommendation and name the replacement: assume a role through OIDC instead of using key X. [2]

Two mistakes: accepting "we rotate the key" as equivalent to removing it; and calling OIDC "secure" without checking the trust policy, which is where the real control lives.

6. Side by side

	IAM user	IAM role
Credentials	access key ID and secret, valid until deleted [2]	none of its own; temporary ones per session [1]
Who uses it	one identity, whoever holds the key	anyone the trust policy allows [1]
Appears in credential report	yes [4]	no, it lists users
Appears in the log as	`user/NAME`	`assumed-role/NAME/SESSION`
Right for the pipeline	only if nothing else works [1]	yes, through federation [2]

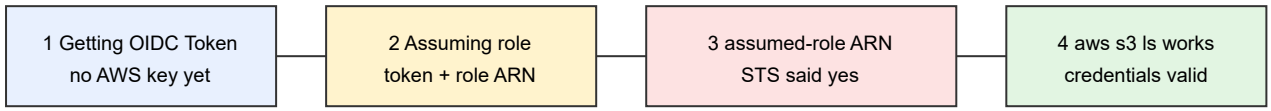
7. Other identities you will meet

- **Root user.** Owns the whole account. AWS says to protect its credentials and require MFA on it. [2]
- **Federated people.** Staff should sign in through an identity provider, ideally IAM Identity Center, not through their own IAM users. [2]
- **Service-linked roles.** Created and owned by an AWS service. You can see them, not edit them. [1]
- **Machines outside AWS.** Roles Anywhere (a certificate) and SAML also give temporary credentials. [2]

8. Worked example: reading the evidence

The captured pipeline log shows four lines. Read them as a chain.

1. "Getting OIDC Token". The service connection asked Azure DevOps for a token. No AWS credential is involved yet. [7]
2. "Assuming role via OIDC Token". The pipeline calls STS with that token and the role ARN. [5]
3. `arn:aws:sts::706569143085:assumed-role/WBG-ADO-ORG-SERVICE-CONNECTION-TEST/aws-vsts-tools`. STS agreed. Account `706569143085`, a role, session `aws-vsts-tools`. [3] [5]
4. A successful `aws s3 ls`. The temporary credentials worked against a resource.



Proves this run used a role. Does not prove no key exists elsewhere.

What this proves for S1: **this run** used no stored key. What it does not prove: that no key exists. A pipeline can succeed through a role while an old key still sits in a variable group. So the S1 evidence is two things, the log above *and* the credential report plus a search of the places keys hide.

Second example, one condition changed. Suppose line 3 instead showed `arn:aws:iam::706569143085:user/ado-deploy`. Now the caller is a user, so a long-term key was used. The role-based design was bypassed. That is a failing S1 even though the pipeline "works". [1] [4]

9. Decoding three ARNs

Same pattern, three answers.

ARN	Reads as
`arn:aws:iam::706569143085:role/WBG-ADO-ORG-SERVICE-CONNECTION-TEST`	the role itself, in account `706569143085` (what the trust policy protects) [3]
`arn:aws:sts::706569143085:assumed-role/WBG-ADO-ORG-SERVICE-CONNECTION-TEST/aws-vsts-tools`	one live session of that role (what shows up in logs) [3] [5]
`arn:aws:iam::706569143085:user/ado-deploy`	an IAM user with a long-term key (what S1 wants gone) [3]

Note `iam` for a stored identity and `sts` for a session.

10. What to ask the team for

You can start the work alone with this list. Each line is one request and what a good answer looks like.

Control	Ask for	Good answer
S1	credential report, `ListAccessKeys` for the old user, a search of variable groups, YAML, scripts and repos	key inactive or user deleted, no key text found [4]
S2	pipeline logs and artifacts from a recent run	no token or session token values visible
A3	the role's trust policy	only `sts:AssumeRoleWithWebIdentity`, only the Azure DevOps provider [5]
A5	the role's permission policy	only the actions and resources the pipeline needs [2]
D1	change tickets for role ARN, service connection and duration	approved by someone other than the author

If a team cannot produce one of these, that gap is itself the finding.

Where this goes next

The same ideas carry other AWS cases: GitHub Actions, SAML sign-in, a cross-account hop. [2] Domain 2 (OIDC and tokens) explains the token that makes the swap possible.

references

1. https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html : role definition, no long-term keys, when to still use a user, service-linked roles.
2. https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html : temporary credentials for workloads, MFA, update and remove keys, ways to deliver credentials outside AWS.
3. https://docs.aws.amazon.com/IAM/latest/UserGuide/reference-arns.html : ARN format, partitions, 12 digit account ID, ARNs are not secret.
4. https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_getting-report.html : credential report columns, four hour limit, permissions, first two keys only.
5. https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRoleWithWebIdentity.html : temporary credentials, three values, duration, no AWS credentials needed for the call.
6. https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers_oidc.html : OIDC federation into a role.
7. https://aws.amazon.com/blogs/modernizing-with-aws/how-to-federate-into-aws-from-azure-devops-using-openid-connect/ : Azure DevOps to AWS with OIDC.